Vue.Js

Criação de um Mural de Eventos para o curso

Por:
Gabriel Richard Zambianchi de Oliveira
Matheus Henrique dos Santos Gomes
Gabriel Gonçalves de Assis Souza
Matheus Mandarini de Menezes

UFMS
A NOSSA UNIVERSIDADE

Contexto histórico do Vue

2014: Evan You, ex-Google, cria o Vue. A ideia era extrair do Angular só o que ele achava mais útil e fazer algo mais leve.

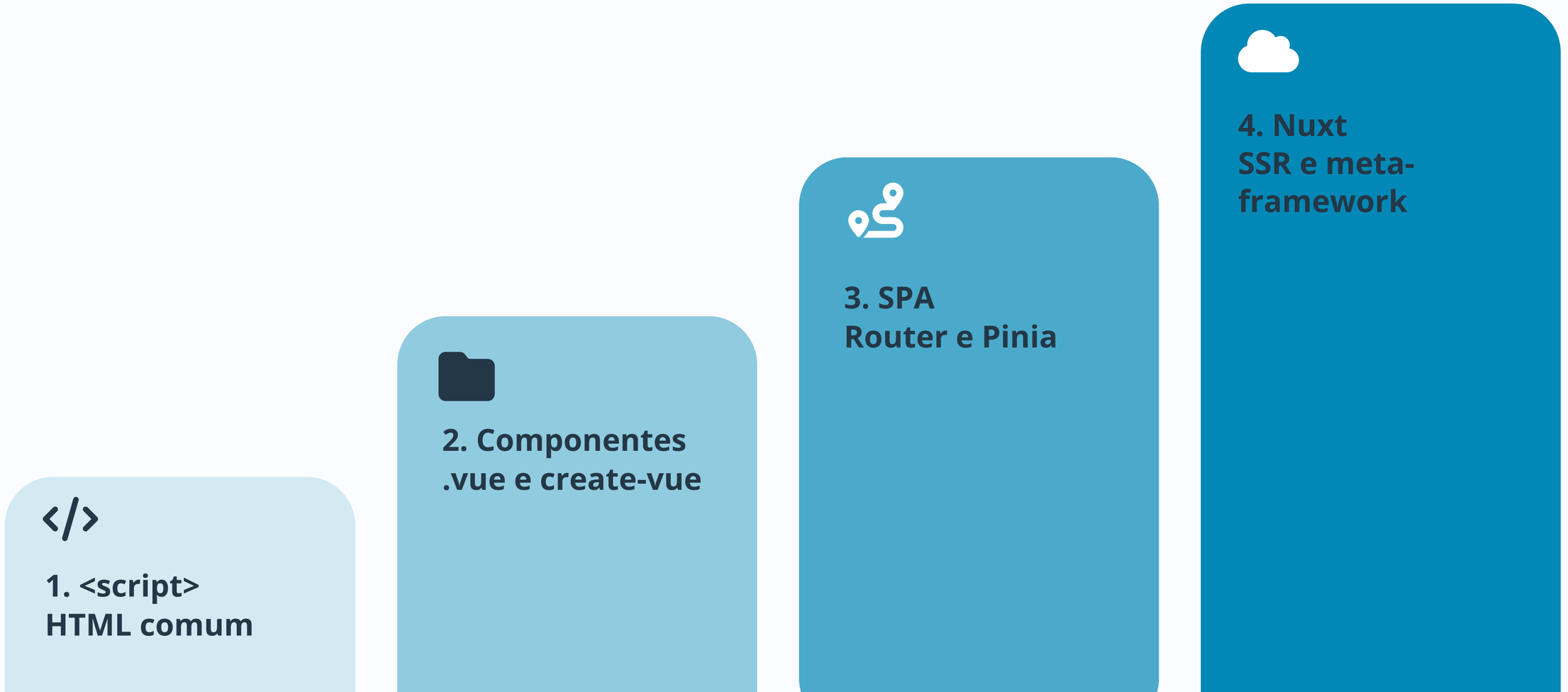
2016: Vue 2 consolida o framework, com virtual DOM e o ecossistema oficial (Vue Router e Vuex).

2020: Vue 3 chega com a Composition API e reatividade baseada em Proxy. No mesmo período surge o Vite, também criado por Evan You.

Atualmente: Pinia substitui o Vuex como solução oficial de estado. O Vue é mantido pela comunidade, sem uma grande empresa por trás.

Filosofia do Framework

Utilize os recursos conforme a necessidade do projeto.



Empresas que utilizam Vue e porquê

Utilizadores

- Alibaba
- GitLab
- Xiaomi
- Nintendo
- Comunidade Laravel

Fonte: vuejs.org (acesso em out. 2026)

Motivação

- Entrada fácil
- Reatividade automática
- Adoção gradual
- Ecossistema oficial coeso

O Framework progressivo

Vue é um framework JavaScript para criar interfaces de usuário.
Ele se baseia em HTML, CSS e JavaScript padrão e fornece um modelo de programação declarativo baseado em componentes que ajuda você a desenvolver com eficiência interfaces de usuário de qualquer complexidade.

Núcleo Nativo

- Biblioteca focada na UI
- Reatividade e Componentes (SFCs)

Ecossistema Oficial

- Escala para SPAs completas
- Pacotes externos mantidos pelo próprio Vue

O que muda na tela ?

O Vue utiliza um sistema de Reatividade Fina, rastreando automaticamente quais variáveis específicas cada parte da interface consome por baixo dos panos.

Quando um estado é alterado, o framework não reconstrói o componente inteiro, ele atualiza cirurgicamente apenas o nó do HTML que depende daquele dado específico.

No **React**, toda a função do componente é **re-executada** a cada mudança de estado.

O **Vue** tem ciência exata das dependências e trabalha focado na alteração mínima necessária.

Estado Local e Global

Gerenciamento de Dados

Estado Local

Composition API
Uso de ref e reactive dentro do componente

Estado Global

Pinia
Solução oficial, moderna e integrada

O ref cuida da tela do componente.
O Pinia cuida daquelas informações que precisam aparecer no site inteiro.

Navegação e Inputs

Roteamento

Para a navegação em Single Page Applications (SPAs), o ecossistema oferece o **Vue Router** como solução oficial e padronizada, garantindo total integração de rotas com o framework.

Formulários

O Vue elimina a necessidade de manipular eventos e valores manualmente com a diretiva **v-model**, que faz a **sincronização automática** (two-way binding) entre a variável do código e o input do usuário em uma única linha.

Templates

HTML estendido

No **React** tudo era **JavaScript (JSX)**.
O **Vue** deixa o desenvolvedor continuar escrevendo o HTML,
adicionando apenas pequenos comandos chamados diretivas (como v-if).

- Não exige JSX por padrão.
- Diretivas de controle (v-if, v-for).
- Separação limpa de lógica e visual

Curva de Aprendizado Suave

Os Single File Components (.vue) mantêm a separação clássica da web que o desenvolvedor já conhece:

- estrutura visual em templates HTML;
- lógica em JavaScript;
- estilos isolados no CSS;

Quando escolher ?

Onde o Vue se destaca ?

Migrações Graduais

Pode ser adotado página por página em sistemas antigos.

Times Multifuncionais

Ideal para equipes que preferem separar CSS, HTML e Lógica.

Média Complexidade

Equilíbrio entre convenção e liberdade.

Criação de Projeto

Estrutura gerada pelo create-vue

- A inicialização de aplicações modernas é feita por meio da ferramenta oficial **create-vue**.
- Utilizado o **Vite** como motor de construção de alta performance
- Gera uma estrutura de pastas pré-configurada, pronta para integrar rotas e gerenciadores de estado global

npm create vue@latest

```
nome-do-projeto/
├── node_modules/
├── public/
│   └── favicon.ico
├── src/
│   ├── assets/
│   ├── components/
│   ├── views/ (se Vue Router for selecionado)
│   ├── router/ (se Vue Router for selecionado)
│   ├── store/ (se Pinia for selecionado)
│   ├── App.vue
│   └── main.js
├── index.html
├── package.json
├── vite.config.js
└── README.md
```

Todo o código passa a viver na pasta **src/**, dividido em Single File Components (SFCs), que unem lógica, estrutura e estilo no mesmo arquivo.

Iniciando um projeto com create-vue

Arquivo **index.html** gerado pelo vue-create

```html
<!DOCTYPE html>
<html lang="">
  <head>
    <meta charset="UTF-8">
    <link rel="icon" href="/favicon.ico">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Vite App</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

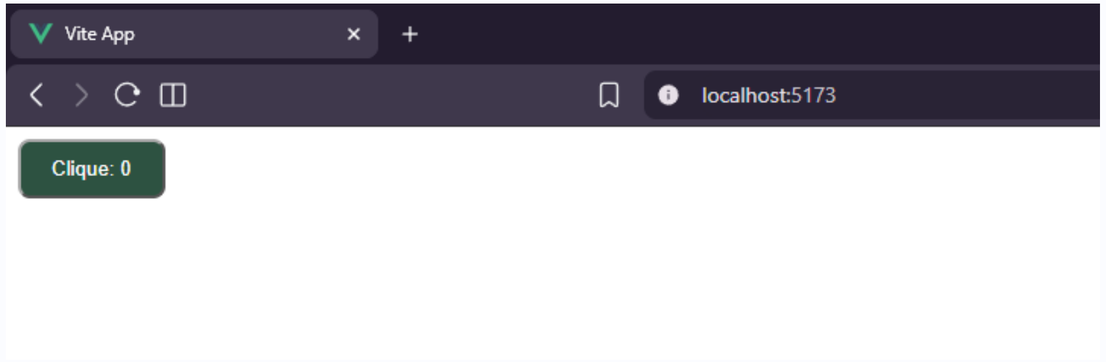
Código do componente

```vue
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">Cliques: {{ count }}</button>
</template>

<style scoped>
button {
  padding: 10px 20px;
  background-color: #42b883;
  color: white;
  border-radius: 8px;
}
</style>
```

Visualização no browser



Single File Components (SFCs)

O Vue organiza tudo em um único arquivo, dividido em três blocos.

- **<script setup>**

  Onde escrevemos o JavaScript. Setup indica o uso da Composition API. Ele expõe as variáveis e funções criadas no JavaScript direto para o <template>

- **<template>**

  Onde escrevemos o HTML. É a interface que o usuário vê.

- **<style scoped>**

  Onde escrevemos o CSS. O termo **scoped** indica ao Vue que esse CSS só vai afetar o HTML que estiver dentro deste exato arquivo, isolando o visual perfeitamente.

```vue
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">Cliques: {{ count }}</button>
</template>

<style scoped>
button {
  padding: 10px 20px;
  background-color: #42b883;
  color: white;
  border-radius: 8px;
}
</style>
```

Reatividade com Ref e Reactive

O Vue gerencia estados reativos através de Proxies, oferecendo duas APIs principais na Composition API:

- **ref() :** Utilizado principalmente para tipos primitivos (Strings, Numbers, Booleans).
  - Encapsula o valor dentro de um objeto reativo.
  - No JavaScript, o acesso e a mutação exigem explicitamente a propriedade .value. No template HTML, o desempacotamento é automático.

```
import { ref } from 'vue'
const contador = ref(0)

contador.value = 1
```

- **reactive() :** Exclusivo para tipos de referência (Objetos, Arrays e Coleções).
  - Retorna um Proxy reativo da estrutura de dados original (objetos com múltiplos atributos).
  - Dispensa o uso de .value; as propriedades são acessadas e mutadas diretamente.

```
import { reactive } from 'vue'
const usuario = reactive({ nome: 'Ana', idade: 25 })

usuario.idade = 26
```

Comparando Ref e Reactive

O **ref** tornou-se a recomendação oficial do Vue por oferecer um padrão mais seguro e universal para qualquer tipo de dado. Oferecendo:

- **Reatribuição Segura:** Você pode substituir os dados por um objeto totalmente novo (ex: usuario.value = novoObjeto) mantendo a reatividade intacta.
- **Padronização:** Usar ref em todo projeto cria uma consistência mental.

O **reactive** acaba sofrendo com uma limitação nativa do JavaScript.

- **A Armadilha da Desestruturação:** Ao extrair propriedades de um objeto reactive para variáveis independentes, o rastreamento do Vue é perdido.
- **A Solução:** Utilizar a função utilitária toRefs(usuario), que transforma cada atributo interno em um ref individual antes da extração.

```
import { reactive, toRefs } from 'vue'

const usuario = reactive({ nome: 'Ana', idade: 25 })

const { idade: idadeReativa } = toRefs(usuario)
```

Reflexão

ref ou reactive: o que acontece?

Desestruturando: perde a reatividade

```
const usuario = reactive({
  nome: 'Ana', idade: 25 })

const { idade } = usuario
// idade é só uma cópia: 25

idade++
// a tela NÃO atualiza
```

Com toRefs: mantém a reatividade

```
const { idade } = toRefs(usuario)
// idade é um ref ligado ao objeto

idade.value++
// a tela atualiza

usuario.idade  // 26
```

- **Por quê:** desestruturar copia o valor primitivo; o Proxy não enxerga mais o acesso à propriedade.
- **toRefs:** cria um ref para cada propriedade, sincronizado com o objeto original.
- **Por isso o ref é o padrão:** o .value deixa explícito onde a reatividade vive.

Reagindo a mudanças: Computed e Watch

Enquanto ref/reactive armazenam os dados, o Vue utiliza o computed e o watch para observar e reagir automaticamente toda vez que esses dados originais sofrem qualquer alteração.

O **Computed** cria uma variável nova derivada de dados originais.

- **Exemplo de uso:** lista filtrada de eventos baseada no texto da busca.
- **Performance (Cache):** memoriza o resultado e só recalcula se a variável original mudar.

```
import { ref, computed } from 'vue'

const busca = ref('rock')
const todosEventos = ref(['Rock in Rio', 'Pop Festival', 'Rock Brasil'])

const eventosFiltrados = computed(() => {
  return todosEventos.value.filter(evento => evento.includes(busca.value))
})
```

Reagindo a mudanças: Computed e Watch

O **Watch** é uma função que fica vigiando uma variável específica para disparar uma ação externa (efeito colateral) sempre que ela sofrer alteração.

O watch não devolve um resultado visual. Ele escuta a mudança na palavra e roda o que estiver dentro da função watch.

```
import { ref, watch } from 'vue'

const busca = ref('rock')

watch(busca, (novoTexto) => {
  console.log(`Novo texto digitado: ${novoTexto}`)
})
```

Template e diretivas

As diretivas são atributos especiais, iniciados por v-, que estendem o HTML com comportamento.

- **v-if / v-else :** Mostra ou remove um elemento conforme uma condição.
- **v-show :** Apenas esconde o elemento com CSS, sem removê-lo.
- **v-for + :key :** Repete um elemento para cada item da lista. A :key identifica cada item.
- **v-bind (:) :** Liga um atributo a um valor do script.
- **v-on (@) :** Escuta eventos, como o clique de um botão.

```
<p v-if="eventos.length === 0">
  Nenhum evento encontrado
</p>

<ul>
  <li v-for="e in eventos" :key="e.id">
    {{ e.titulo }}
  </li>
</ul>

<button @click="salvar">Salvar</button>
```

v-model e Two-Way Binding

O v-model sincroniza o campo do formulário com uma variável, nos dois sentidos.

- **Dado → campo :** quando a variável muda, o valor mostrado no campo é atualizado.
- **Campo → dado :** quando a pessoa digita, a variável é atualizada.
- **Comparando com o React :** o formulário controlado exige value e onChange em cada campo. No Vue, o v-model faz as duas coisas em uma linha.
- **Atenção :** a conveniência esconde o fluxo de dados. Por isso é importante saber o que acontece por baixo.

VUE

```
<input v-model="busca">
```

O QUE O VUE FAZ POR BAIXO

```
<input :value="busca"
  @input="busca = $event.target.value">
```

REACT

```
<input value={busca}
  onChange={(e) => setBusca(e.target.value)} />
```

Reflexão

v-model: o custo de clareza

- **O que parece mágica:** o v-model usa propriedade e evento diferentes conforme o elemento.
- **Custo:** quem lê o código não vê de onde o dado entra e sai.
- **Benefício:** bem menos código repetido em formulários grandes.
- **Modificadores:** .trim, .number e .lazy ajustam o comportamento.

Conhecer o desdobramento evita bugs no fluxo de dados.

Comunicação entre Componentes

Os componentes se comunicam em uma direção definida: os dados descem e os eventos sobem.

- **definePropS()** : O componente pai envia dados ao filho.
  - O filho apenas lê a prop. Ele não deve alterá-la.
- **defineEmits()** : O filho avisa o pai que algo aconteceu, emitindo um evento.
  - O pai escuta o evento com @.
- **provide / inject** : Para dependências distantes na árvore, sem passar props por cada nível.

NO FILHO (CardEvento.vue)

```
const props = defineProps(['evento'])
const emit = defineEmits(['interesse'])

emit('interesse', props.evento.id)
```

NO PAI (ListaEventos.vue)

```
<CardEvento
  :evento="e"
  @interesse="alternar"
/>
```

Vue Router

O Vue Router é a solução oficial de rotas: liga cada endereço a um componente, sem recarregar a página.

- **RouterLink :** Link de navegação que troca de página sem recarregar.
- **RouterView :** Mostra o componente da rota atual.
- **useRoute() :** Lê informações da rota atual, como o parâmetro route.params.id.
- **useRouter() :** Navega por código, como router.push('/').

CONFIGURAÇÃO: UMA TELA PARA CADA ENDEREÇO

```
const routes = [
  { path: '/', component: ListaEventos },
  { path: '/eventos/novo',
    component: FormEvento },
  { path: '/eventos/:id',
    component: DetalheEvento },
]
```

USO NO COMPONENTE: LER O ID E NAVEGAR

```
const route = useRoute()
route.params.id

const router = useRouter()
router.push('/')
```

Pinia: Estado Global

O Pinia é a solução oficial de estado global: uma store compartilhada por todos os componentes.

- **defineStore() :** Cria a store. No estilo setup, usa os mesmos ref e computed dos componentes.
  - ref guarda o estado (a lista de ids de interesse).
  - computed calcula valores derivados (o total).
  - Funções alteram o estado (alternar).
- **Uso :** qualquer componente chama a store e lê ou altera os mesmos dados, sem relação de pai e filho.

CRIANDO A STORE (src/stores/interesses.js)

```
export const useInteresses =
  defineStore('interesses', () => {
    const ids = ref([])

    const total = computed(
      () => ids.value.length
    )

    function alternar(id) { /* ... */ }

    return { ids, total, alternar }
  })
```

USO NO CABEÇALHO, CARD E DETALHE

```
const interesses = useInteresses()

<p>Meus interesses: {{ interesses.total }}</p>
```

Options API e Ecossistema

A Options API é a sintaxe anterior do Vue e ainda aparece em muitos projetos existentes.

- **data() :** Estado do componente (equivale ao ref e ao reactive).
- **computed e methods :** Valores derivados e funções do componente.
- **this :** Acessa os dados dentro dos métodos. Na Composition API, não existe this.
- **Nuxt :** Meta-framework do Vue, que acrescenta SSR, SSG e convenções de pastas.

OPTIONS API

```
export default {
  data() {
    return { contador: 0 }
  },
  computed: {
    dobro() { return this.contador * 2 }
  },
  methods: {
    somar() { this.contador++ }
  }
}
```

COMPOSITION API

```
<script setup>
const contador = ref(0)
const dobro = computed(() => contador.value * 2)
function somar() { contador.value++ }
</script>
```

Mural de Eventos do Curso

RF1 - Listar por data, com carregamento e erro

RF2 - Buscar no título e filtrar por categoria

RF3 - Cadastrar com validação de campos

RF4 - Abrir detalhe e tratar evento inexistente

RF5 - Marcar interesse com contador global

LIVE CODING

RF1: busca e estados da lista

Criar a página inicial, que mostra todos os eventos do curso em cartões, do mais próximo ao mais distante. Enquanto os eventos chegam, a pessoa vê um aviso de que está carregando. Se algo der errado, vê uma mensagem clara.

Raciocínio da implementação

- Buscar GET /eventos ao montar.
- Ordenar pela data crescente.
- Exibir cards com título, data, local e categoria.
- O título aponta para /eventos/:id.

LIVE CODING

RF2: busca e categoria combinadas

Vamos ajudar a pessoa a achar um evento mais rápido. Ela escolhe uma categoria e digita parte do título, e a lista muda enquanto ela digita. Se nenhum evento combinar, aparece “Nenhum evento encontrado”.

Controles do usuário

- Busca ligada por v-model.
- Seletor: Todas e quatro categorias. Filtro calculado com computed.
- A lista muda a cada tecla.

Oficina: 45 minutos em duplas

Agora é com vocês!!!

Preparação



Oficina

Checkpoint 1: cadastro de evento

RF3

Objetivo

Criar a tela para cadastrar um evento novo (/eventos/novo). A pessoa preenche o formulário VIA POST /eventos e, se estiver tudo certo, o evento aparece na lista.

Passo a Passo:

1. Criar **form** com **reactive** e ligar cada campo com **v-model**
2. Calcular os erros com um **computed**
3. Mostrar cada erro abaixo do campo **(v-if + aria-describedby)**
4. **@submit.prevent**: enviar com **fetch** (POST) se não houver erros
5. Redirecionar com **useRouter().push('/')**

Está pronto quando:

- Não é possível enviar o formulário com campos obrigatórios vazios ou com data passada;
- O novo evento aparece na lista após o cadastro;
- Todos os campos têm <label> associado.

Tempo: 18 min · Solução: branch checkpoint-1

Oficina

Checkpoint 2: detalhe do evento

RF4

Objetivo

Criar a página de um evento só /eventos/:id. Ao clicar no título de um evento, a pessoa vê todos os detalhes dele obtidos de GET /eventos/:id.

Passo a Passo:

1. Registrar a rota **/eventos/:id** em **src/router/index.js**
2. Ler o id com **useRoute().params.id**
3. Buscar o evento e usar **watch** no id para buscar de novo se ele mudar
4. Tratar: carregando, encontrado e inexistente
5. Botão “Voltar” com **RouterLink**

Está pronto quando:

- O título do card leva ao detalhe sem recarregar a página;
- Acessar /eventos/999 mostra “Evento não encontrado”.

Tempo: 12 min · Solução: branch checkpoint-2

Oficina

Checkpoint 3: interesses com Pinia

RF5

Objetivo

Deixar a pessoa marcar os eventos de que gostou. Uma “caixa” compartilhada guarda quais eventos têm interesse. No topo da página, aparece quantos eventos ela marcou.

Passo a Passo:

1. Criar **src/stores/interesses.js** com **defineStore**
2. Guardar os ids em **ref([])** e expor **alternar(id), temInteresse(id) e total (computed)**
3. Usar a store no cabeçalho, nos cards e no detalhe
4. Abrir as Vue DevTools e ver a aba Pinia

Está pronto quando:

- O contador do cabeçalho muda imediatamente ao clicar no botão de um card;
- O estado se mantém ao navegar entre lista e detalhe.

Tempo: 15 min · Solução: branch checkpoint-3

Quiz!!!!

Kahoot!

5 perguntas

Pergunta-chave

Qual a diferença entre um computed e um método chamado no template?

Duas formas de calcular

```
const numero = ref(1)
const cliques = ref(0)

const dobro = computed(() => {
  console.log('computed')
  return numero.value * 2
})

function dobroMetodo() {
  console.log('metodo')
  return numero.value * 2
}
```

Um contador independente

```
<p>{{ dobro }}</p>
<p>{{ dobroMetodo() }}</p>
<p>Cliques: {{ cliques }}</p>

<button @click="numero++">
  Mudar numero
</button>

<button @click="cliques++">
  Outro estado
</button>
```

Quem executa de novo, e por quê???

Pergunta-chave - Resposta

Qual a diferença entre um computed e um método chamado no template?

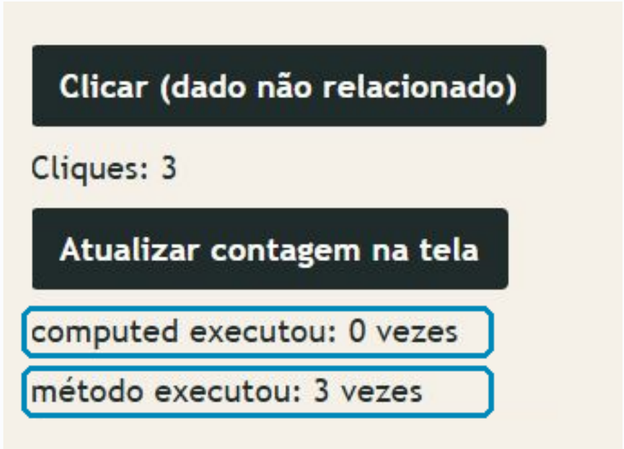
AO CLICAR EM “OUTRO ESTADO”
Só o método executa de novo.

POR QUÊ
O computed guarda o resultado (cache) e só recalcula quando o dado que ele usa, o numero, muda.
O método roda toda vez que a tela é desenhada.

O que aparece no console

Ação	computed	método
Abrir a página	executa	executa
Clicar em “Mudar número”	executa	executa
Clicar em “Outro estado”	**não executa**	executa

Na prática: computed x método



No console

```
Console was cleared                                    DemoComputedMetodo.vue:48
4 método filtrar() executou                            DemoComputedMetodo.vue:33
```

Só o método aparece. O computed não executou nenhuma vez: usou o resultado em cache.

Síntese

Item	Vue
Natureza	Framework progressivo: núcleo de UI; Vue Router e Pinia oficiais
Templates	HTML com diretivas (v-if, v-for, v-model)
Reatividade	Automática: o Vue rastreia quem usa cada dado
Estado local e global	ref e reactive (local); Pinia (global)
Rotas e formulários	Vue Router (oficial); formulário com v-model
Criação de projeto	create-vue (Vite): src/ com components, views, router e stores
Curva de aprendizado	Suave no início, mais conceitos com o tempo
Quando escolher	Adoção gradual e equipes que vêm de HTML e CSS

Dúvidas?

Repositório do grupo:

GitHub

Referências

VUE.JS. Guia oficial do Vue 3. Disponível em: vuejs.org/guide. Acesso em: 30 out. 2026.

VUE.JS. Guia oficial do Vue 3 (em português). Disponível em: pt.vuejs.org/guide. Acesso em: 30 out. 2026.

REACT. Learn React. Disponível em: react.dev/learn. Acesso em: 30 out. 2026.

REACT. React Reference Overview. Disponível em: react.dev/reference/react. Acesso em: 30 out. 2026.

VUE ROUTER. Vue Router: guia oficial. Disponível em: router.vuejs.org/guide. Acesso em: 30 out. 2026.

PINIA. Pinia: the intuitive store for Vue. Disponível em: pinia.vuejs.org. Acesso em: 30 out. 2026.

NUXT. Nuxt Documentation. Disponível em: nuxt.com/docs. Acesso em: 30 out. 2026.

VUE DEVTOOLS. Vue DevTools. Disponível em: devtools.vuejs.org. Acesso em: 30 out. 2026.

VUE.JS. Vue art: logos e identidade visual (repositório GitHub). Disponível em: github.com/vuejs/art. Acesso em: 30 out. 2026.